

Correct but Fragile: Numerical Stability Risks in Machine Learning Systems

Cordell Stonecipher

January 30, 2026

Project Title

Correct but Fragile: Numerical Stability Risks in Machine Learning Systems

Course

CSI 4150 – AI for IT Operations

Project Description

Machine learning systems deployed in operational environments are commonly evaluated using accuracy and convergence metrics. While these indicators confirm functional correctness, they do not guarantee numerical robustness, reproducibility, or deployment reliability. This project investigates how machine learning models that appear correct can still exhibit numerical instability under realistic operational conditions. The focus is on identifying silent reliability failures that emerge during training, deployment, and serving phases, and on demonstrating how MLOps tooling can be used to detect and manage these risks.

Problem Statement

Machine learning pipelines may successfully train, pass validation, and be deployed, yet still behave unpredictably when subjected to common operational variations such as hardware changes, mixed-precision execution, or configuration drift. These failures do not generate explicit errors but manifest as inconsistent predictions, non-deterministic behavior, or sensitivity to minor environmental changes. Such behavior poses a significant challenge for AI-driven IT operations, where consistency, reliability, and reproducibility are critical.

Project Goal

The goal of this project is to demonstrate that numerical instability represents a hidden operational risk in machine learning systems and to evaluate how MLOps practices can be used to expose and manage this risk. The project aims to show that models can maintain stable accuracy while still exhibiting fragile numerical behavior across operational contexts.

Relevance to AI for IT Operations

This project aligns directly with AI for IT Operations by framing numerical instability as an operational reliability issue rather than a modeling flaw. The study focuses on reproducibility, environment consistency, artifact reliability, and deployment robustness. These concerns are central to operating AI systems in production IT environments, where failures often occur silently rather than as explicit system crashes.

Proposed Methodology

A controlled machine learning pipeline will be implemented using a fixed dataset, model architecture, and hyperparameters. The baseline system will be executed under multiple operational conditions without modifying model logic. These conditions include variations in random seeds, hardware backends (CPU versus GPU), numerical precision (FP32 versus FP16/AMP), batch sizes, and data preprocessing order. Each experiment run will be tracked and versioned to ensure reproducibility and enable systematic comparison across configurations.

MLOps Tools and Operational Integration

To align with course objectives, the project will integrate the following tools throughout development and experimentation:

- **Git** for source control and traceability of code and configuration changes.
- **GitHub** for repository hosting and collaboration.
- **GitHub Actions** for CI-style automated runs that execute experiment matrices and regression checks.
- **Docker** to standardize and reproduce execution environments across machines and hardware backends.
- **Conda** to manage Python dependencies locally and to mirror the Docker environment when needed.
- **MLflow** for experiment tracking (run parameters, metrics, artifacts, and stability indicators).

- **DVC** for dataset and artifact versioning (data snapshots and model outputs tied to Git commits).
- **PyTorch** for model implementation and controlled precision experiments (FP32 vs AMP).

The tools are used as operational mechanisms to reveal hidden fragility: CI automation (GitHub Actions) repeatedly executes controlled perturbations, while tracking (MLflow) and versioning (Git/DVC) ensure results are reproducible and comparable across runs.

System Diagram

Figure 1 illustrates the end-to-end pipeline. A fixed dataset and preprocessing stage feeds a training component that runs under an explicit configuration matrix (seed, precision, batch size, device). Outputs are logged and versioned; a stability evaluation module computes robustness metrics and triggers CI-style pass/fail signals based on reproducibility and output agreement thresholds.

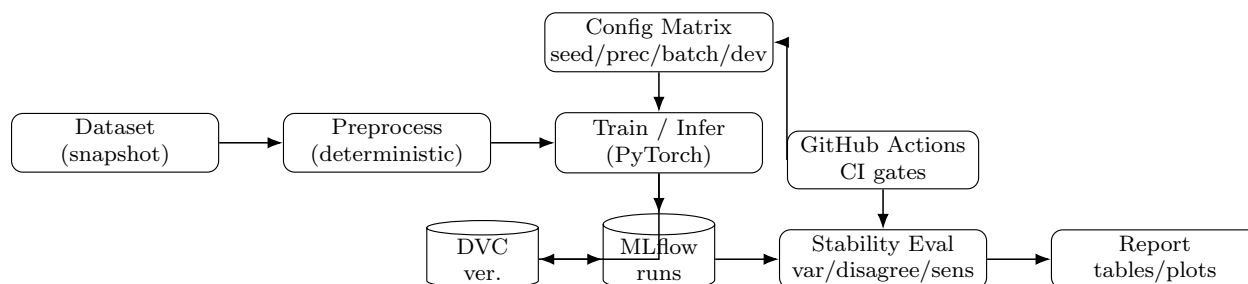


Figure 1: Compact pipeline for detecting numerical instability using an MLOps/DevOps toolchain.

Evaluation Criteria

Model performance will be evaluated using standard accuracy metrics to confirm functional correctness. Additional operational metrics will include prediction variance across runs, disagreement rates between model artifacts, sensitivity to small input perturbations, gradient norm behavior, and reproducibility gaps across identical configurations. All metrics will be logged and analyzed through MLflow, with datasets and artifacts versioned via Git/DVC and validated via CI automation.

Expected Outcomes

The expected outcome is evidence that machine learning models can remain accurate while exhibiting numerically fragile behavior under realistic operational conditions. The project is expected to show that conventional validation practices may fail to detect these issues unless numerical robustness is explicitly monitored and enforced through CI gates.